

▼ BPL_TEST2_Batch script with OMSimulator

The key library OMSimulator is installed.

After the installation a small application BPL_TEST2_Batch is loaded and run. You can continue with this example if you like.

```
!lsb_release -a # Actual VM Ubuntu version used by Google
```

```
No LSB modules are available.
Distributor ID: Ubuntu
Description:    Ubuntu 24.04.5 LTS
Release:        24.04
Codename:       noble
```

```
!python --version
```

```
Python 3.13.15
```

```
!pip install OMSimulator
```

```
Collecting OMSimulator
  Downloading omsimulator-3.0.0.post116.tar.gz (7.6 kB)
  Installing build dependencies ... done
  Getting requirements to build wheel ... done
  Preparing metadata (pyproject.toml) ... done
Requirement already satisfied: requests in /usr/local/lib/python3.13/dist-packages (from OMSimulator) (2.32.4)
Requirement already satisfied: lxml>=5.3.1 in /usr/local/lib/python3.13/dist-packages (from OMSimulator) (6.1.3)
Requirement already satisfied: charset_normalizer<4,>=2 in /usr/local/lib/python3.13/dist-packages (from requests) (3.4.1)
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.13/dist-packages (from requests) (3.10.1)
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.13/dist-packages (from requests) (2.3.0)
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.13/dist-packages (from requests) (2025.11.12)
Building wheels for collected packages: OMSimulator
  Building wheel for OMSimulator (pyproject.toml) ... done
  Created wheel for OMSimulator: filename=omsimulator-3.0.0.post116-py3-none-any.whl size=1345325 sha256=68d283d3e1b1b52e2f4f54edb8db1eb3f7a8a579620c69b88
  Stored in directory: /root/.cache/pip/wheels/eb/b0/ee/5194adbce75a2efc0262c5c0377b28ecbc648628be707b5a4
Successfully built OMSimulator
Installing collected packages: OMSimulator
Successfully installed OMSimulator-3.0.0.post116
```

```
!pip install DyMat
```

```
Collecting DyMat
  Downloading DyMat-0.7.tar.gz (24 kB)
  Preparing metadata (setup.py) ... done
Building wheels for collected packages: DyMat
  Building wheel for DyMat (setup.py) ... done
  Created wheel for DyMat: filename=dymat-0.7-py3-none-any.whl size=19077 sha256=5934c77ce8b3aec0ba5aa2f613a3576e
  Stored in directory: /root/.cache/pip/wheels/bd/8e/7a/d63cada988615a826edc5353c397a21762db0633d8372d7e3
Successfully built DyMat
Installing collected packages: DyMat
Successfully installed DyMat-0.7
```

```
!pip install git+https://github.com/janpeter19/FMU_explore.git
```

```
Collecting git+https://github.com/janpeter19/FMU_explore.git
  Cloning https://github.com/janpeter19/FMU_explore.git to /tmp/pip-req-build-5esn7wjf
  Running command git clone --filter=blob:none --quiet https://github.com/janpeter19/FMU_explore.git /tmp/pip-req-build-5esn7wjf
  Resolved https://github.com/janpeter19/FMU_explore.git to commit a11b52e2f4f54edb8db1eb3f7a8a579620c69b88
  Installing build dependencies ... done
  Getting requirements to build wheel ... done
  Preparing metadata (pyproject.toml) ... done
Building wheels for collected packages: fmu_explore
  Building wheel for fmu_explore (pyproject.toml) ... done
  Created wheel for fmu_explore: filename=fmu_explore-1.2.5-py3-none-any.whl size=29342 sha256=b1c9fa16db55e50ecf
  Stored in directory: /tmp/pip-ephem-wheel-cache-lptl19m5/wheels/98/33/d9/8669158b0d845b084c49212b2330ad684e92d7
Successfully built fmu_explore
Installing collected packages: fmu_explore
Successfully installed fmu_explore-1.2.5
```

▼ BPL_TEST2_Batch setup

Now specific installation and the run simulations. Start with connecting to Github. Then upload the two files:

- FMU - BPL_TEST2_Batch_linux_om_me.fmu
- Setup-file - BPL_TEST2_Batch_fmpy_explore.py

```
%%bash
git clone https://github.com/janpeter19/BPL_TEST2_Batch
```

Cloning into 'BPL_TEST2_Batch'...

```
%cd BPL_TEST2_Batch
```

```
/content/BPL_TEST2_Batch
```

```
# Import the application model and context variables
from BPL_TEST2_Batch_oms import*
```

Linux – run FMU pre-compiled OpenModelica

```
# Import the general FMU_explore functions ana adapt them to the application
from fmu_explore_oms import AdaptWith
```

```
Application = AdaptWith(parValue, parLocation, parCheck, \
                          fmu_model, fmu_process_diagram, \
                          MSL_usage, MSL_version, BPL_version, \
                          simulationTime, timeDiscreteStates, \
                          diagrams, ax, lines)
```

```
par = Application.par
init = Application.init
restart = Application.restart
```

```
resetPen = Application.resetPen
process_diagram = Application.process_diagram
```

```
describe_MSL = Application.describe_MSL
FMU_explore_info = Application.FMU_explore_info
system_info = Application.system_info
SDG = Application.SDG
```

```
run -i BPL_TEST2_Batch_oms_explore.py
```

Model for the process has been setup. Key commands:

- par() - change of parameters and initial values
- init() - change initial values only
- simu() - simulate and plot
- newplot() - make a new plot
- show() - show plot from previous simulation
- disp() - display parameters and initial values from the last simulation
- describe() - describe culture, broth, parameters, variables with values/units

Note that both disp() and describe() takes values from the last simulation and the command process_diagram() brings up the main configuration

Brief information about a command by help(), eg help(simu)
Key system information is listed with the command system_info()

Basic simulation

```
restart()
```

```
Application.model.modelName
```

```
'BPL.Examples_TEST2.Batch'
```

```
for s in Application.model.states:
    print(s.name)
```

```
bioreactor.V
bioreactor.m[1]
bioreactor.m[2]
```

```
Application.model.instantiate()
```

```
Application.model.setResultFile('result.mat')
```

Start coding or [generate](#) with AI.

```
Application.model.setValue('bioreactor.V_start', 1.0)
Application.model.setValue('bioreactor.m_start[1]', 1.0)
Application.model.setValue('bioreactor.m_start[2]', 10.0)
```

```
Application.model.initialize()
```

```
Application.model.simulate()
```

```
mat_data = scipy.io.loadmat('result.mat')
```

```
mat_data.keys()
```

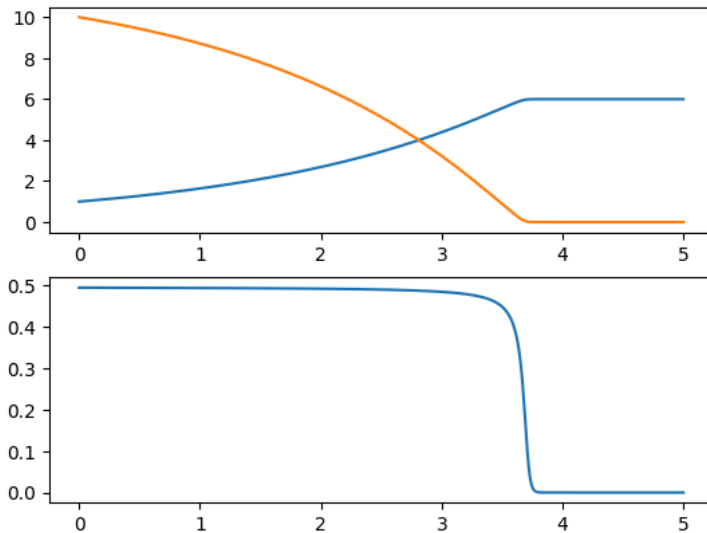
```
dict_keys(['Aclass', 'name', 'description', 'dataInfo', 'data_1', 'data_2'])
```

```
t = mat_data['data_2'][0];
```

```
plt.figure()
```

```
ax1 = plt.subplot(2,1,1)
ax2 = plt.subplot(2,1,2)
```

```
ax1.plot(t, mat_data['data_2'][2])
ax1.plot(t, mat_data['data_2'][3])
ax2.plot(t, mat_data['data_2'][12])
plt.show()
```



Simulation looks what I expected!

```
Application.model.terminate()
Application.model.delete()
```

```
Application.model.delete()
```

✓ Parameter sweep and simulation

```
def simu():

    diagrams = Application.diagrams
    ax = Application.ax
    linecycler = Application.linecycler

    parValue = Application.parValue
    parLocation = Application.parLocation

    model = Application.model

    # Prepare for simulation
    Application.restart()
    Application.model.setResultFile('result.mat')
```

```
# Set parameters
Application.model.setValue('bioreactor.V_start', 1.0)
Application.model.setValue('bioreactor.m_start[1]', 1.0)
Application.model.setValue('bioreactor.m_start[2]', value)

# Simulation
Application.model.initialize()
Application.model.simulate()

# Open up result file
mat_data = scipy.io.loadmat('result.mat')
t = mat_data['data_2'][0];

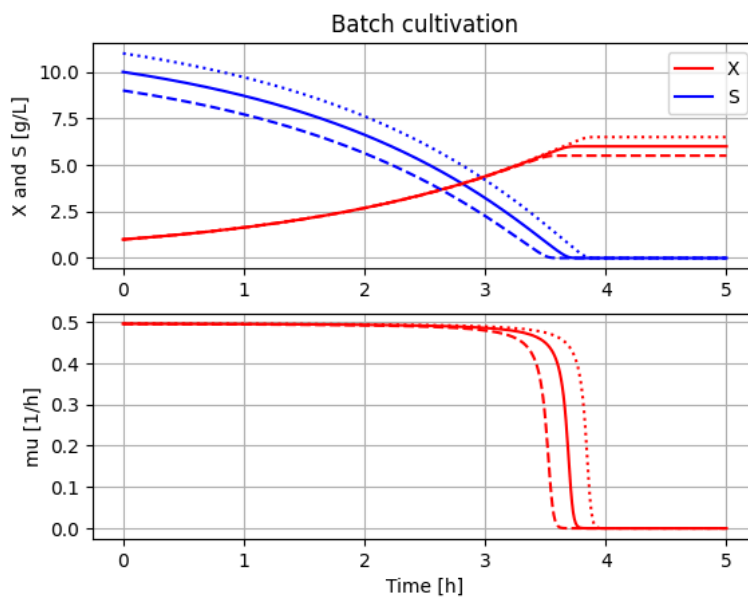
# Plot results
linetype = next(linecycler)
context = locals().copy()

for command in diagrams:
    eval(command, context)

#Application.ax[0].plot(t, mat_data['data_2'][2])
#Application.ax[0].plot(t, mat_data['data_2'][3])
#Application.ax[1].plot(t, mat_data['data_2'][12])

# Close the model
Application.model.terminate()
Application.model.delete()
```

```
newplot()
for value in [10, 9, 11]:
    simu()
```



```
newplot()

# Parameter sweep
for value in [10,9,11]:

    # Prepare for simulation
    Application.restart()
    Application.model.setResultFile('result.mat')

    # Set parameters
    Application.model.setValue('bioreactor.V_start', 1.0)
    Application.model.setValue('bioreactor.m_start[1]', 1.0)
    Application.model.setValue('bioreactor.m_start[2]', value)

    # Simulation
    Application.model.initialize()
    Application.model.simulate()

    # Open up result file
    mat_data = scipy.io.loadmat('result.mat')
```

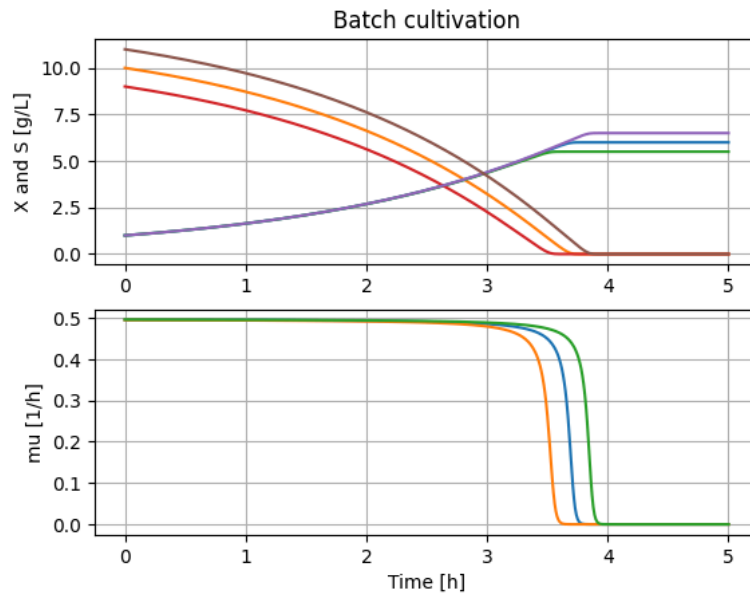
```

t = mat_data['data_2'][0];

# Plot results
Application.ax[0].plot(t, mat_data['data_2'][2])
Application.ax[0].plot(t, mat_data['data_2'][3])
Application.ax[1].plot(t, mat_data['data_2'][12])

# Close the model
Application.model.terminate()
Application.model.delete()

```



▼ How to address variables using names

```
res = DyMat.DyMatFile('result.mat')
```

```
print(res.names())
```

```
plt.figure()
```

```
ax1 = plt.subplot(2,1,1)
ax2 = plt.subplot(2,1,2)
```

```
ax1.plot(t, res['Batch.bioreactor.m[1]'])
ax1.plot(t, res['Batch.bioreactor.m[2]'])
ax2.plot(t, res['Batch.bioreactor.culture.mu'])
plt.show()
```

```
newplot()
```

```

-----
NameError                                Traceback (most recent call last)
/content/BPL_TEST2_Batch/BPL_TEST2_Batch_oms_explore.py in <cell line: 0>()
----> 1 newplot()

/content/BPL_TEST2_Batch/BPL_TEST2_Batch_oms_explore.py in newplot(title, plotType)
    26         diagram = 'PhasePlane'
    27
----> 28     resetPen()
    29
    30     # Plot diagram

NameError: name 'resetPen' is not defined

```

How to address "time" in the res-file?

